RAPPORT TECHNIQUE
PHASE 1

Étude et préparation d'un système de génération de descriptions d'images
Approche RAG – Groupe G23

Dataset : MS COCO 2017
Préparation des données • Exploration • YOLO • CLIP • FAISS • Évaluation initiale

Réalisé par : Groupe G23
Année universitaire : 2026–2027

Document élaboré à partir du dépôt du projet lp_project-main et de ses résultats enregistrés.

Résumé

Ce rapport présente les travaux réalisés durant la première phase du projet du groupe G23, consacré à l’étude des approches Retrieval-Augmented Generation (RAG) appliquées à la génération automatique de descriptions d’images (Image Captioning). La démarche repose sur le dataset MS COCO 2017. Cette phase a principalement pour objectif de constituer un corpus exploitable, d’en vérifier la qualité, d’en analyser les caractéristiques visuelles et textuelles, puis de construire une première base de connaissances vectorielle permettant de tester la recherche de similarité.

Le corpus retenu comprend 11 100 images réparties en trois ensembles : 10 000 images d’entraînement, 100 images de validation et 1 000 images de test. Les images et les captions ont été prétraitées. Les représentations vectorielles ont été extraites avec CLIP ViT-B/32, puis indexées avec FAISS à l’aide de l’index IndexFlatIP. Une première évaluation du retrieval a été menée sur 98 images de validation. Les résultats obtenus constituent une base expérimentale initiale ; ils doivent être interprétés avec prudence en raison de la définition de pertinence retenue et ne représentent pas encore une évaluation complète du mécanisme Corrective RAG.

Mots-clés : Image Captioning, RAG, Corrective RAG, MS COCO, YOLO, CLIP, Embeddings, FAISS, Retrieval.

Table des matières

- 1. Introduction et objectifs
- 2. Présentation du dataset
- 3. Sélection et constitution des ensembles
- 4. Téléchargement et validation
- 5. Exploration et analyse statistique
- 6. Détection d’objets avec YOLO
- 7. Contrôle qualité et prétraitement
- 8. Extraction des embeddings avec CLIP
- 9. Construction de la base de connaissances avec FAISS
- 10. Tests et évaluation du retrieval
- 11. Organisation du projet et reproductibilité
- 12. Limites et perspectives
- 13. Conclusion

1. Introduction et objectifs

La génération automatique de descriptions d'images consiste à produire une phrase en langage naturel décrivant le contenu visuel d'une image. Cette tâche combine des problématiques de vision par ordinateur et de traitement automatique du langage naturel : le système doit identifier les éléments présents, comprendre certains liens entre eux et formuler une description cohérente.

Le projet s'inscrit dans le cadre des architectures Retrieval-Augmented Generation. Une architecture RAG associe une étape de recherche d'informations pertinentes (retrieval) à une étape de génération (generation). L'approche Corrective RAG (CRAG) ajoute une évaluation de la qualité des informations récupérées et prévoit des mécanismes de correction lorsque le contexte obtenu est insuffisant ou peu fiable.

L'objectif de la Phase 1 n'est pas encore de revendiquer un système CRAG complet. Il s'agit de préparer les données et les composants techniques nécessaires aux phases ultérieures : corpus propre, annotations analysées, représentations multimodales, index vectoriels et premiers tests de recherche.

2. Présentation du dataset

Le projet utilise MS COCO 2017 (Microsoft Common Objects in Context), un dataset de référence en vision par ordinateur. Il contient des images de scènes quotidiennes, des annotations d'instances d'objets et des descriptions rédigées par des annotateurs humains. Les annotations d'instances couvrent 80 catégories d'objets.

L'intérêt de COCO pour ce projet est double : les annotations permettent d'analyser les objets et la complexité des scènes, tandis que les captions fournissent des descriptions textuelles pouvant être associées aux images. Cette complémentarité est particulièrement utile pour une approche multimodale fondée sur la recherche image-texte.

3. Sélection et constitution des ensembles

Un sous-ensemble de 11 100 images a été constitué à partir de COCO 2017. Le manifeste split_manifest.csv conserve les identifiants, les noms de fichiers, les URL COCO et l'affectation de chaque image.

Ensemble	Nombre d'images	Source	Utilisation
Train	10 000	train2017	Construction de la base de connaissances
Validation	100	train2017	Tests et évaluation pendant le développement
Test	1 000	val2017	Évaluation finale réservée
Total	11 100	COCO 2017	Corpus du projet

La séparation des ensembles vise à éviter le data leakage. Les images de validation et de test ne sont pas intégrées à la base FAISS. Le choix déterministe des images, associé à une graine aléatoire fixe (seed = 42, selon le script de sélection), facilite la reproductibilité.

4. Téléchargement et validation

Le téléchargement a été automatisé dans les scripts de preprocessing. Le traitement utilise des requêtes HTTP et un mécanisme de téléchargement concurrent afin de réduire le temps d'attente. Une logique de

nouvelle tentative permet de gérer les erreurs réseau temporaires. Les fichiers téléchargés sont ensuite vérifiés afin de s'assurer qu'ils sont lisibles.

Indicateur de téléchargement	Résultat
Images prévues	11 100
Images téléchargées	11 099
Images ignorées car déjà présentes	1
Téléchargements échoués	0

Le rapport de validation ne signale aucune anomalie : les 11 100 fichiers attendus sont présents et lisibles, aucune image manquante ou corrompue n'est rapportée. Le fichier validation_report.csv est vide, ce qui indique qu'aucune anomalie n'a été consignée.

5. Exploration et analyse statistique

5.1 Distribution des catégories COCO

L'analyse des annotations fait apparaître 80 267 instances d'objets dans le sous-ensemble. Les catégories les plus fréquentes comprennent person (24 534 instances), car (4 077), chair (3 612), book (2 260) et bottle (2 233). La catégorie person domine nettement la distribution.

Ces valeurs correspondent à des instances d'objets et non au nombre d'images : une même image peut contenir plusieurs objets d'une même catégorie. Cette analyse met en évidence une distribution déséquilibrée des catégories, élément à prendre en compte lors de l'interprétation des expériences.

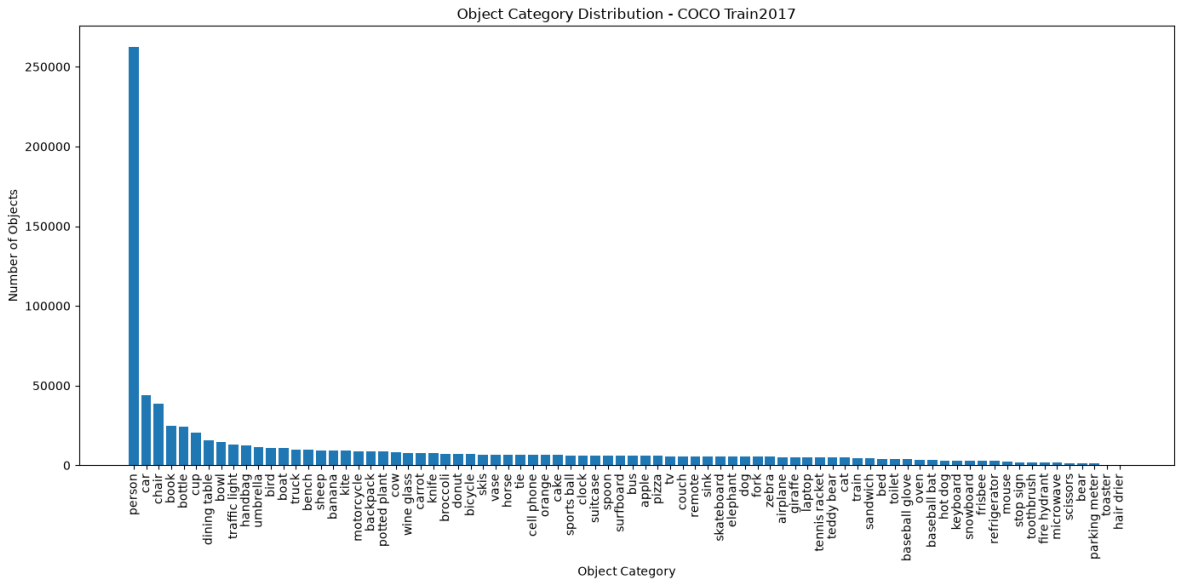


Figure 1 — Distribution des catégories d'objets selon les annotations COCO.

5.2 Métadonnées des images

L'analyse porte sur les dimensions, le ratio largeur/hauteur et la taille des fichiers. La largeur moyenne est de 576,47 pixels et la hauteur moyenne de 483,46 pixels. La taille moyenne d'un fichier est de 159,50 KB, avec un minimum de 8,92 KB et un maximum de 585,38 KB. La résolution la plus fréquente est 640 × 480 pixels.

Cette diversité justifie une étape de normalisation du format et de vérification de l'orientation, tout en conservant les dimensions originales à ce stade du prétraitement.

5.3 Analyse des captions

Le fichier d'analyse contient 55 526 captions originales. La longueur moyenne est de 10,46 mots, la médiane de 10 mots, le minimum de 6 mots et le maximum de 47 mots. Cette variabilité reflète des niveaux de détail différents : certaines descriptions se limitent à l'action ou à l'objet principal, tandis que d'autres mentionnent davantage d'éléments de la scène.

5.4 Nombre d'objets par image et complexité des scènes

Le nombre moyen d'objets annotés par image est de 7,23 sur l'ensemble du corpus. Il est de 7,2062 pour Train, 6,9 pour Validation et 7,515 pour Test. Le maximum observé est de 56 objets dans une image.

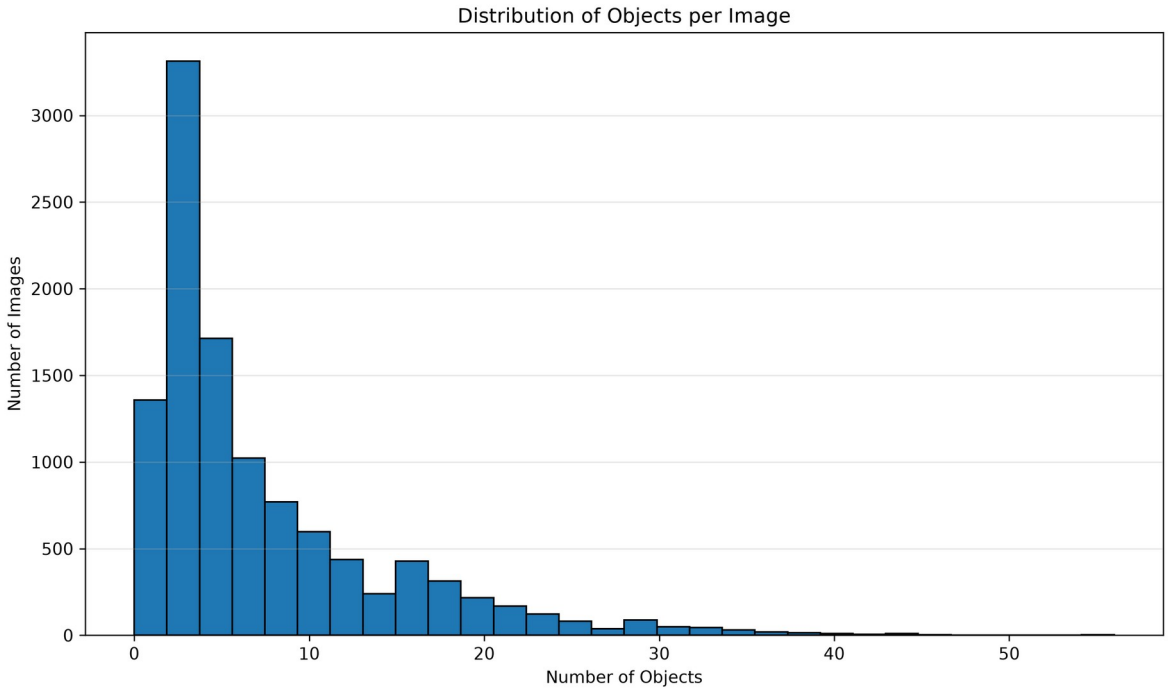


Figure 2 — Histogramme du nombre d'objets annotés par image.

Pour faciliter l'analyse, les scènes ont été regroupées selon des seuils définis pour le projet : Simple (0 à 3 objets), Medium (4 à 10 objets) et Complex (plus de 10 objets). On obtient 4 671 images simples (42,08 %), 3 823 images moyennes (34,44 %) et 2 606 images complexes (23,48 %). Ces seuils sont des choix analytiques propres au projet et ne constituent pas des catégories officielles de COCO.

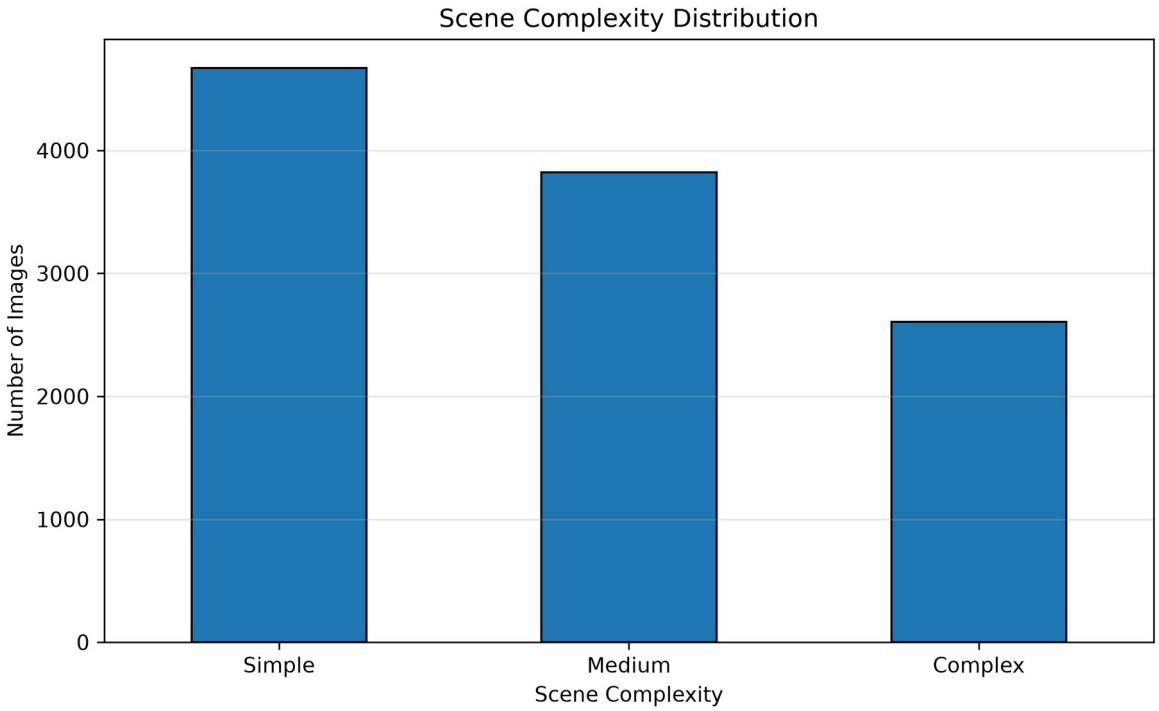


Figure 3 — Répartition des niveaux de complexité selon les annotations COCO.

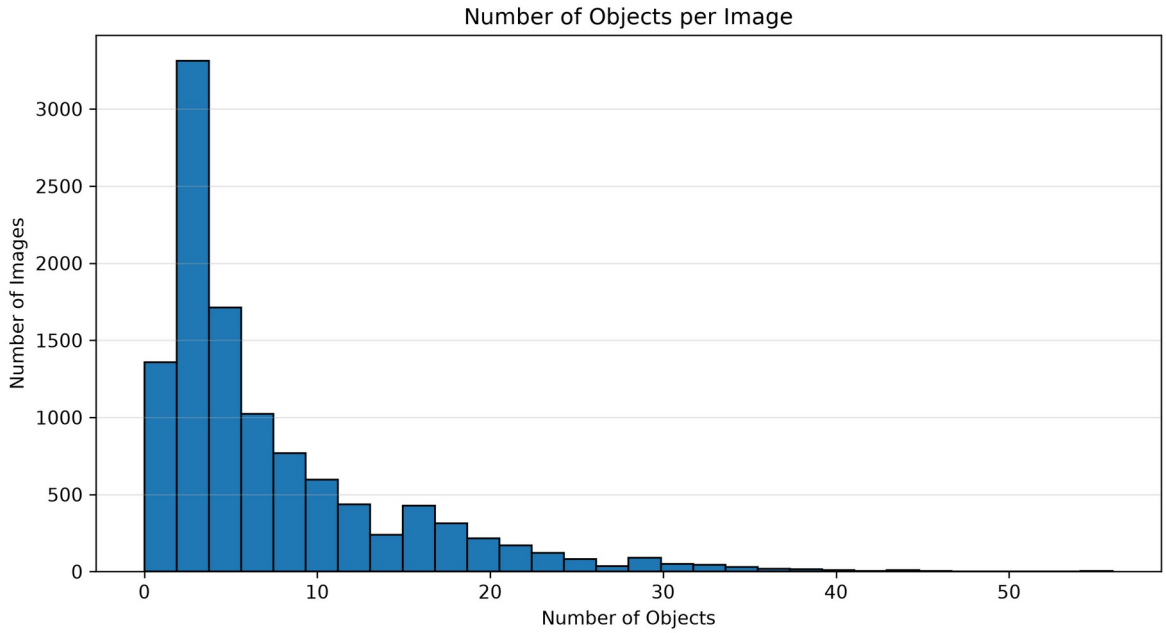


Figure 4 — Distribution du nombre d'objets et seuils de complexité.

6. Détection d'objets avec YOLO

En complément des annotations COCO, le projet exécute YOLO11n sur les images afin d'obtenir des détections calculées directement à partir des pixels. Le modèle utilisé est YOLO11n, avec une taille d'entrée de 640 pixels, un seuil de confiance de 0,25, un batch size de 8 et une exécution sur CPU.

Le fichier yolo_detections.csv contient 60 832 détections. Le nombre moyen de détections est de 5,48 par image (5,47 pour Train, 5,35 pour Validation et 5,64 pour Test). 171 images ne comportent aucune détection enregistrée. Les catégories les plus détectées sont person (20 805), car (3 128), chair (2 217), bottle (1 467) et cup (1 342).

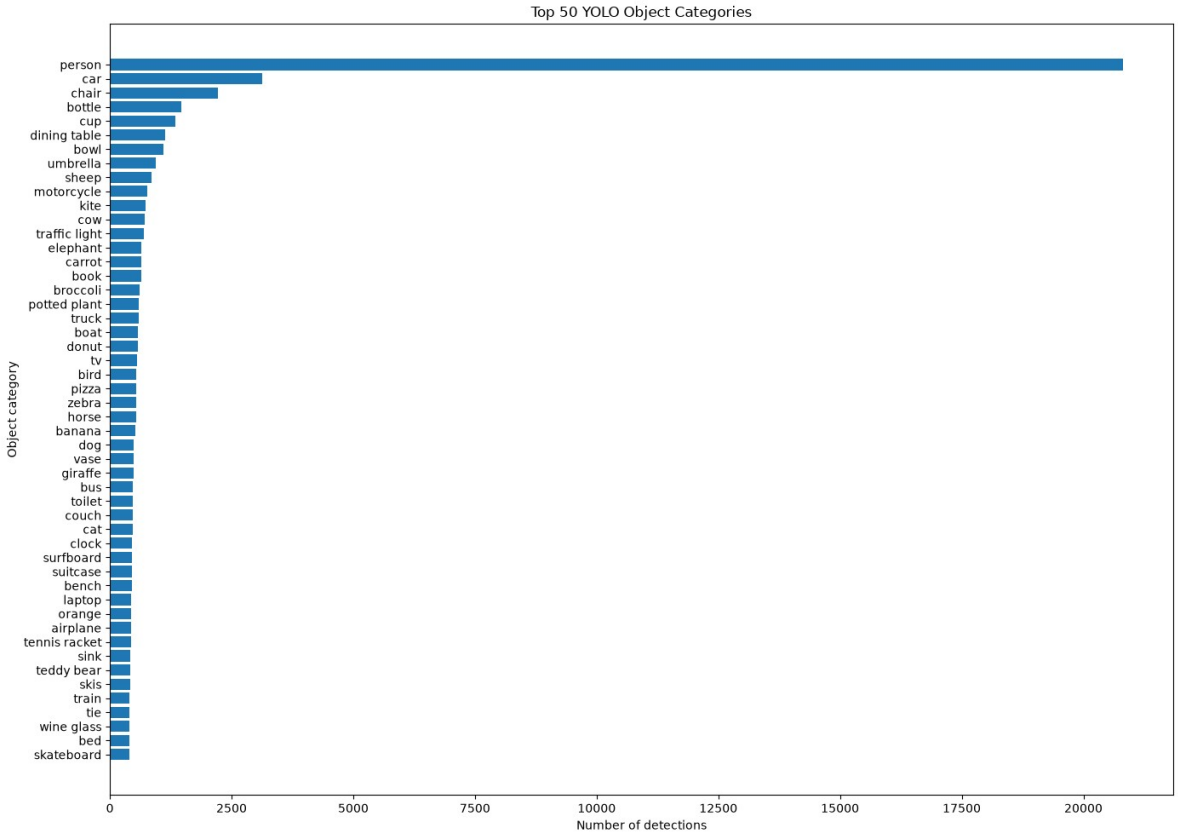


Figure 5 — Les catégories les plus fréquemment détectées par YOLO.

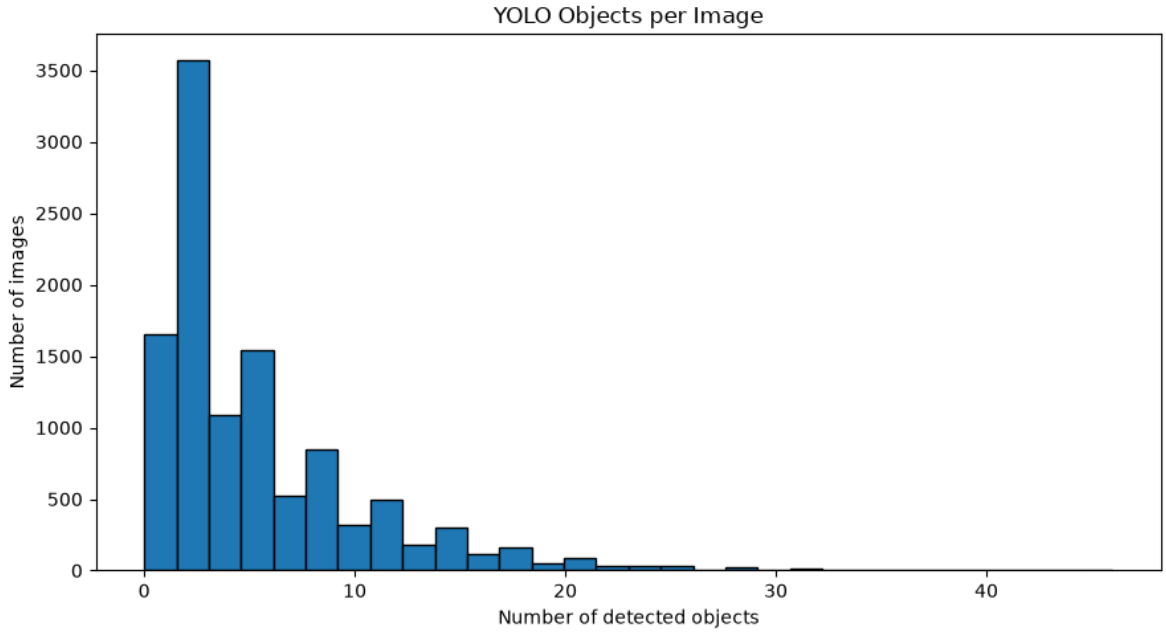


Figure 6 — Nombre de détections YOLO par image.

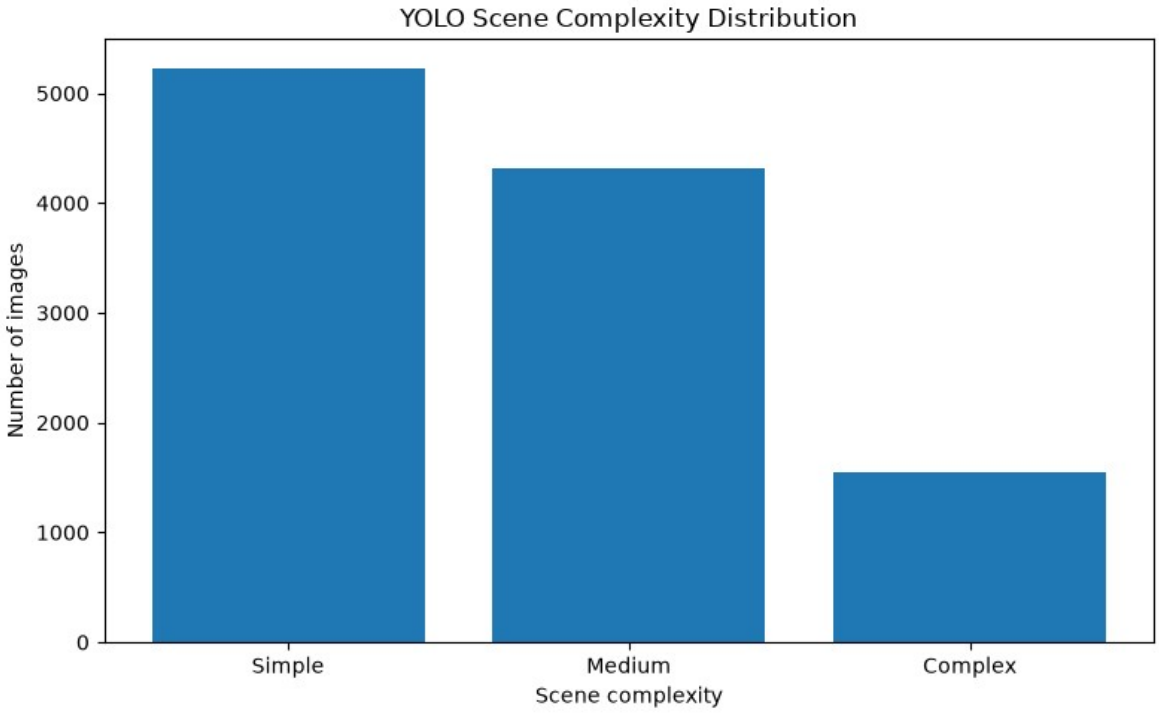


Figure 7 — Répartition de la complexité calculée à partir des détections YOLO.

Les annotations COCO et les prédictions YOLO ne doivent pas être confondues. Les premières servent de référence annotée ; les secondes sont des sorties de modèle et peuvent comporter des faux positifs ou des objets manqués. Les différences entre leurs distributions ne constituent donc pas, à elles seules, une mesure de précision du détecteur. Une évaluation formelle de YOLO nécessiterait notamment une comparaison détection par détection avec les boîtes de vérité terrain et des métriques telles que la précision, le rappel et l'IoU.

7. Contrôle qualité et prétraitement

7.1 Contrôle de qualité

Le contrôle qualité vérifie l'unicité des images, l'absence de chevauchement entre les ensembles, la disponibilité des captions et la cohérence des fichiers. Les résultats enregistrés indiquent 11 100 images dans le corpus, aucun groupe de doublons détecté et aucun chevauchement signalé entre Train, Validation et Test. La majorité des images possède cinq captions, même si le fichier original comporte 55 526 descriptions.

7.2 Prétraitement des images

Le script preprocess_images.py applique la correction de l'orientation EXIF, convertit les images en RGB, puis les enregistre au format JPEG avec une qualité de 95 et l'option d'optimisation. Les dimensions originales sont conservées dans cette étape. Le rapport image_preprocessing_report.csv contient 11 100 lignes avec le statut success et aucune erreur enregistrée.

7.3 Prétraitement des captions

Le script preprocess_captions.py effectue une normalisation textuelle : passage en minuscules, suppression de la ponctuation et normalisation des espaces. Le texte original est conservé en parallèle du

texte nettoyé. Cinq captions sont sélectionnées de manière déterministe par image pour les fichiers de travail.

Fichier / ensemble	Nombre de captions
Captions originales	55 526
Captions traitées	55 500
Train	50 000
Validation	500
Test	5 000

La différence entre 55 526 et 55 500 s'explique par la sélection de cinq captions par image : 11 100 × 5 = 55 500. Les captions de validation et de test sont conservées pour l'évaluation, mais seules les captions Train alimentent l'index textuel de la base de connaissances.

8. Extraction des embeddings avec CLIP

Le modèle openai/clip-vit-base-patch32 est utilisé pour encoder les images et les textes dans un espace vectoriel commun. CLIP (Contrastive Language–Image Pre-training) comporte un encodeur d'image et un encodeur de texte. Chaque entrée est représentée par un vecteur de 512 dimensions.

Cette représentation commune permet de mesurer la proximité sémantique entre une image et une description. Les embeddings sont normalisés avec la norme L2, de sorte que leur norme soit approximativement égale à 1. Cette normalisation rend possible l'utilisation du produit scalaire comme équivalent de la similarité cosinus.

Type d'embedding	Forme (shape)	Nombre de vecteurs
Images Train	(10 000, 512)	10 000
Images Validation	(100, 512)	100
Images Test	(1 000, 512)	1 000
Captions Train	(50 000, 512)	50 000

Les vecteurs de validation et de test sont générés pour les expériences et l'évaluation, mais ils ne sont pas insérés dans la base de connaissances. Le contrôle des embeddings vérifie notamment les dimensions, la normalisation et l'absence de valeurs non finies.

9. Construction de la base de connaissances avec FAISS

FAISS (Facebook AI Similarity Search) est utilisé pour indexer les vecteurs et retrouver rapidement les voisins les plus proches. Le projet utilise IndexFlatIP, un index fondé sur le produit scalaire (Inner Product). Puisque les vecteurs ont été normalisés par L2, le produit scalaire entre deux vecteurs correspond à leur similarité cosinus.

La base contient 10 000 embeddings d'images Train et 50 000 embeddings de captions Train, accompagnés de leurs métadonnées. Le fichier knowledge_base_info.json confirme que seules les données Train sont indexées et que les ensembles Validation et Test ne sont pas intégrés à l'index.

10. Tests et évaluation du retrieval

10.1 Test qualitatif

Un test de recherche a été réalisé à partir d'une image de validation dont l'identifiant est 14151. Le résultat image le mieux classé présenté dans les résultats du projet porte l'identifiant 132576, avec un score de

similarité de 0,8212. La caption textuelle la mieux classée est : "A man riding skis off the side of a ramp in the snow.", avec un score de 0,3358.

Ces scores sont des mesures de proximité dans l'espace vectoriel. Ils ne doivent pas être interprétés comme des pourcentages de précision ou de probabilité.

10.2 Évaluation quantitative

L'évaluation enregistrée porte sur 98 images de validation sur 100. Deux images n'ont pas pu être évaluées selon la règle de pertinence définie, faute de catégories COCO adaptées à ce calcul.

Métrique	Valeur	Interprétation
Recall@5	0,004516	Part des éléments pertinents retrouvés dans les cinq premiers résultats, selon la règle utilisée.
MRR	0,962925	Moyenne du rang réciproque du premier résultat jugé pertinent.
Similarité moyenne	0,792525	Score moyen de similarité des résultats récupérés.

La règle de pertinence enregistrée considère qu'une image récupérée est pertinente si elle partage au moins une catégorie d'objet COCO avec l'image requête. Cette définition est permissive : deux images peuvent contenir une catégorie commune sans représenter la même action, le même contexte ou la même scène.

Les valeurs de Recall@5 et de MRR doivent être lues conjointement et en tenant compte de cette règle. Un Recall@5 très faible et un MRR élevé peuvent refléter des comportements différents du protocole de recherche ou de calcul des métriques ; ils ne suffisent pas à conclure que le système est performant. Il est recommandé de vérifier l'implémentation exacte des métriques, d'examiner des exemples pertinents/non pertinents et de définir une pertinence plus fine avant toute conclusion.

11. Organisation du projet et reproductibilité

Le dépôt sépare les scripts de traitement, les résultats et les métadonnées. Le dossier preprocessing regroupe les scripts de sélection, téléchargement, validation, analyse, détection, prétraitement, génération d'embeddings, construction de l'index et évaluation. Le dossier results est organisé en figures, statistiques, embeddings et knowledge_base.

Cette organisation facilite la traçabilité des étapes et la réexécution des traitements. Les fichiers CSV conservent les métadonnées et les résultats tabulaires, les figures illustrent les analyses, et le fichier JSON décrit les paramètres essentiels de la base FAISS.

Le dépôt fourni contient également un fichier .gitignore. Les données volumineuses et certains artefacts générés sont exclus du suivi Git, ce qui permet de limiter la taille du dépôt. Le rapport décrit l'état des fichiers présents dans l'archive fournie ; la procédure exacte de publication et l'état distant du dépôt GitHub ne peuvent pas être vérifiés à partir de cette seule archive.

12. Limites et perspectives

- Les résultats actuels correspondent à une première chaîne de retrieval et non à une implémentation complète de Corrective RAG.
- Les métriques de retrieval reposent sur une règle de pertinence fondée sur le partage d'au moins une catégorie COCO ; cette règle ne garantit pas une similarité sémantique complète.
- Les scores de similarité ne sont pas des pourcentages d'exactitude.

- Les prédictions YOLO n'ont pas été présentées ici comme une évaluation quantitative du détecteur contre les annotations Ground Truth.
- Le corpus est un sous-ensemble de COCO 2017 ; les conclusions ne doivent pas être généralisées automatiquement à l'ensemble du dataset ou à d'autres domaines.

Pour la suite, il sera pertinent d'implémenter le mécanisme de correction CRAG : évaluer la pertinence du contexte récupéré, déclencher une recherche complémentaire ou une stratégie alternative lorsque le contexte est faible, puis fournir le contexte validé au composant de génération. L'évaluation devra inclure des métriques de qualité de captions adaptées, une comparaison avec une approche RAG de base et une analyse qualitative des erreurs.

13. Conclusion

La Phase 1 a permis de constituer un corpus de 11 100 images MS COCO 2017, de vérifier son intégrité, d'étudier ses caractéristiques visuelles et textuelles, puis de préparer les données pour une recherche multimodale. L'analyse des annotations COCO et des prédictions YOLO a apporté deux perspectives complémentaires sur le contenu des images. Le prétraitement des captions, l'extraction des embeddings CLIP et leur indexation dans FAISS ont abouti à une première base de connaissances composée de 10 000 vecteurs d'images et de 50 000 vecteurs textuels.

Les premiers tests confirment que la chaîne technique de représentation et de recherche est en place. Les métriques obtenues doivent toutefois être interprétées avec prudence et approfondies à l'aide d'un protocole de pertinence mieux défini. Cette base constitue le point de départ des travaux de la phase suivante, consacrée à l'intégration et à l'évaluation du mécanisme Corrective RAG pour la génération de descriptions d'images.

Annexe — principaux scripts du projet

Script	Rôle principal
select_images.py	Sélection des images et création du manifeste des splits.
download_images.py / download_dataset.py	Téléchargement des images COCO.
validate_dataset.py	Vérification de la présence et de la lisibilité des images.
analyze_categories.py	Analyse des catégories COCO.
analyze_image_metadata.py	Analyse des dimensions et tailles des images.
analyze_captions.py	Analyse statistique des descriptions.
analyze_objects_per_image.py	Comptage des objets annotés par image.
analyze_scene_complexity.py	Classification descriptive de la complexité des scènes.
extract_objects_yolo.py	Détection d'objets avec YOLO11n.
preprocess_images.py	Normalisation des images.
preprocess_captions.py	Nettoyage et sélection des captions.
test_clip.py / generate_embeddings.py	Test de CLIP et génération des vecteurs.
check_embeddings.py	Contrôle des embeddings.
build_knowledge_base.py	Construction des index FAISS.
test_retrieval.py / evaluate_retrieval.py	Tests qualitatifs et évaluation du retrieval.